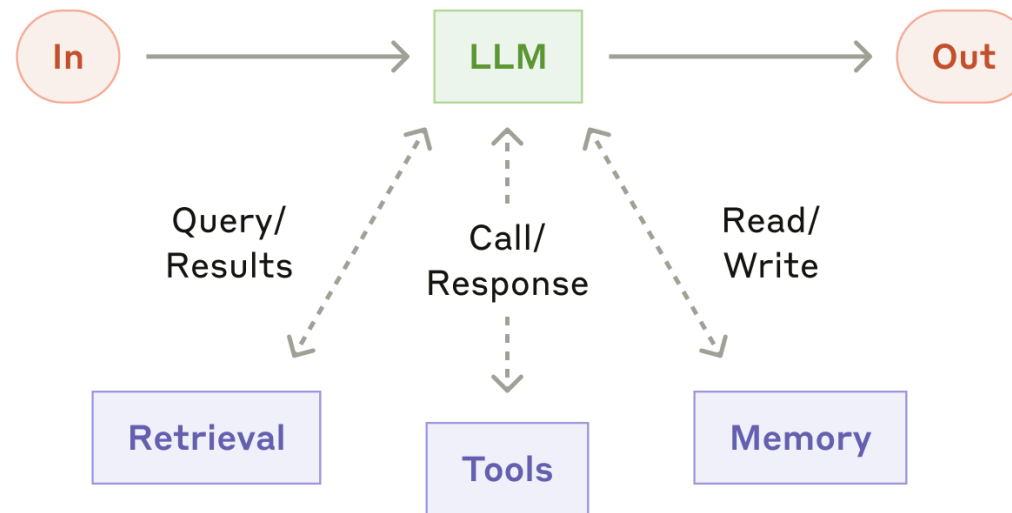


Workshop: Agentic Engineering

Thomas Hein — Dataciders

Agenda



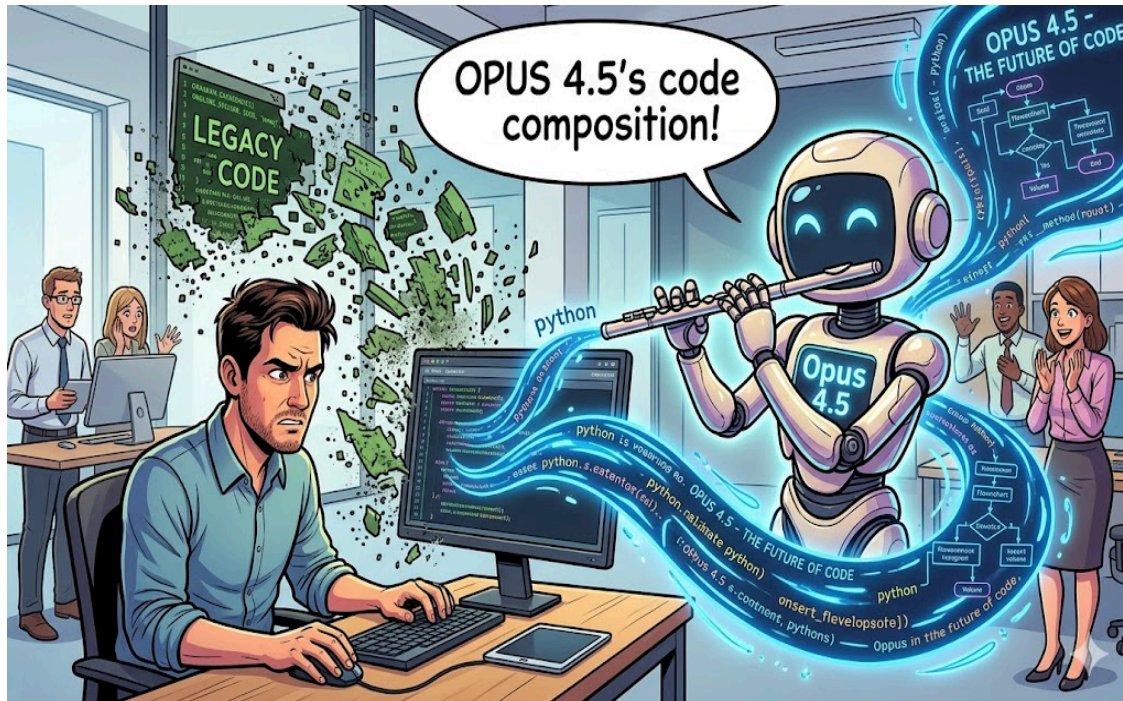
Block 1: Grundlagen & Orientierung

Workshop: Agentic Engineering

Thomas Hein — Datacidars

Meine Motivation

24.11.2025 - Der Tag an dem Coding starb



Opus 4.5 steht der Welt zur Verfügung

Blick in die Zukunft — Was bleibt fuer uns übrig?

- Wer von euch findet gut, was gerade in der Softwareentwicklung passiert?
- Was sind Chancen und Risiken

Die grosse Debatte

"Nobody has to program. The programming language is human."
— Jensen Huang, 2024

"This is the best time yet to learn to code."
— Bill Gates, 2025

"You are orchestrating agents who do and acting as oversight."
— Andrej Karpathy, 2026

Zahlen und Fakten

Unternehmen	AI-generierter Code	Quelle
Google	>30%	Pichai, Q1 2025
Microsoft	20-30%	Nadella, Apr 2025
Meta	Ziel: 50% bis 2026	Zuckerberg, 2025

- **Gartner:** 90% der Enterprise-Entwickler nutzen AI bis 2028
- **Aber:** 2500% mehr Defekte durch unkontrollierte AI-Nutzung



Erkenntnis: Es wird mehr Software geben und diese Software ist fehlerhafter

Was ist eigentlich ein Agent? — Vom GPT zum Agenten

Sprachmodelle sind Wahrscheinlichkeitsmaschinen

Ein LLM vervollständigt immer den nächsten Token — gesteuert durch Kontext:

Eingabe	Mögliche Fortsetzung
Das ist eine Geschichte über	Tiere, Freundschaft, Drachen
Das ist eine Firma in	Hennigsdorf, Berlin, München
def calculate_tax(income):	return income * 0.19

- Kein Verständnis, keine Intention — nur nächster wahrscheinlichster Token
- Qualität hängt ab von: **Trainingsdaten + Kontext (Prompt)**

Was ist eigentlich ein Agent? — Das Chat-Protokoll

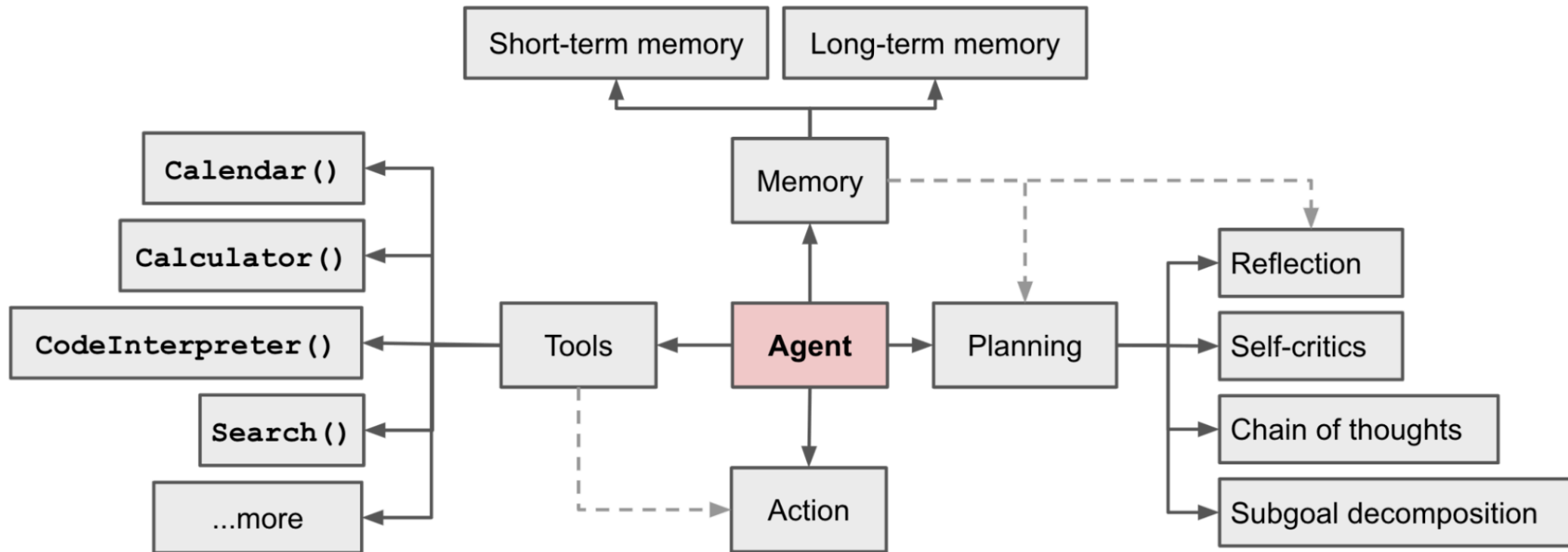
Wie Entwickler mit LLMs arbeiten: strukturierter Kontext

```
[  
  { "role": "system", "content": "Du bist ein hilfreicher Assistent..." },  
  { "role": "user", "content": "Schreibe einen Brief an den Chef..." },  
  { "role": "assistant", "content": "Der Brief..." },  
  { "role": "user", "content": "Erweitere den Text nach dem Protokoll" }  
]
```

- **System-Prompt:** Rolle, Kontext, Regeln, Wissen (AGENTS.md, CLAUDE.md)
- **User:** Aufgabe oder Folgefrage
- **Assistant:** Antwort des Modells — wird Teil des nächsten Kontexts

Das Protokoll ist der eigentliche Hebel — wer den Kontext kontrolliert, steuert den Agenten.

Was ist eigentlich ein Agent? — Architektur



Quelle: Lilian Weng — "LLM Powered Autonomous Agents" (2023)

Was ist eigentlich ein Agent? — Der Agenten-Loop

Ein AI Agent = LLM + Werkzeuge + Schleife

```
tools = [search_web, read_file, run_tests, write_code]

while not task_done:
    response = llm.call(messages, tools=tools) # LLM entscheidet
    if response.tool_call:
        result = execute_tool(response.tool_call) # Werkzeug ausführen
        messages.append(result) # Ergebnis in Kontext
    elif response.is_input_needed:
        user_input = ask_human(response.question) # Mensch gefragt
        messages.append(user_input) # Human-in-the-Loop
    elif response.is_reflecting:
        messages.append(response.reflection) # Selbstkorrektur
    else:
        task_done = True # Aufgabe abgeschlossen
```

Was ist Agentic Engineering nicht

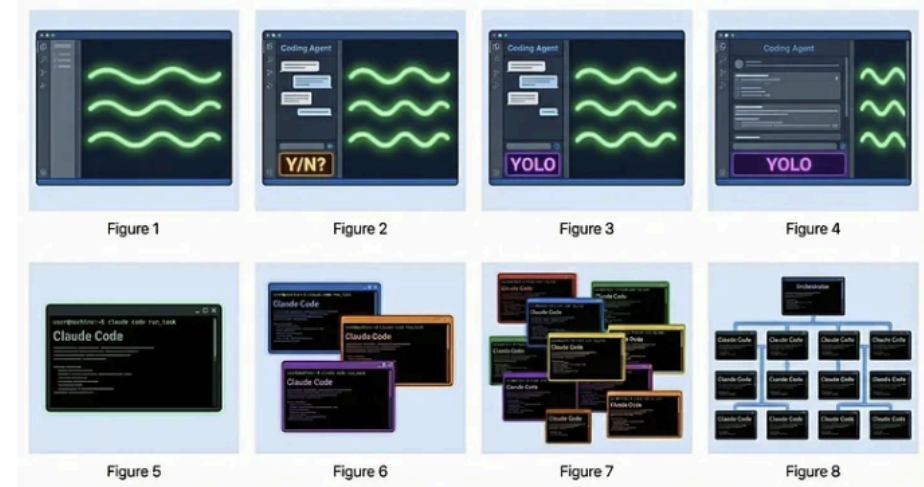
#	Ansatz	Input	Menschliche Rolle
1	Vibe Coding	Kurze Phrasen, fachliche Wuensche	Fachanwender
2	Spec-Driven (Business)	Pflichtenheft, Use Cases	IT Consultant
3	Spec-Driven (Technical)	UML, ERM, Architektur	Architekt

Definition Agentic Engineering

Agentic Engineering ist eine Softwareentwicklungsmethode, bei der Menschen KI-Agenten orchestrieren, um Code zu planen, zu schreiben, zu testen und bereitzustellen – unter strukturierter menschlicher Aufsicht.

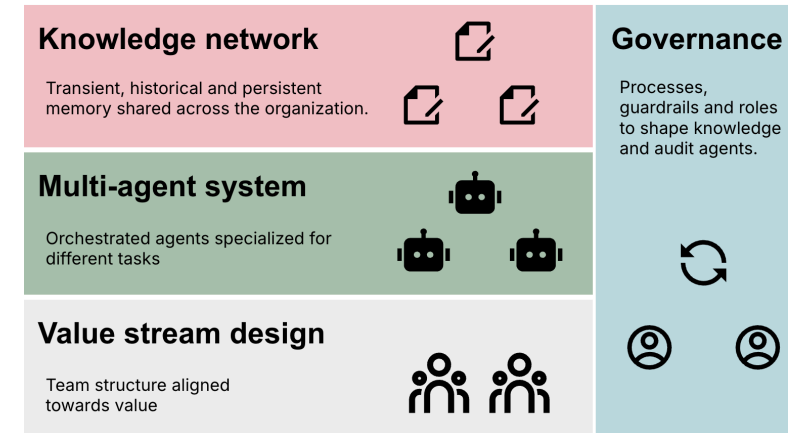
- Steve Wilson

Steve Yegge - Good Agile, Bad Agile



Änderungen im Arbeitsumfeld

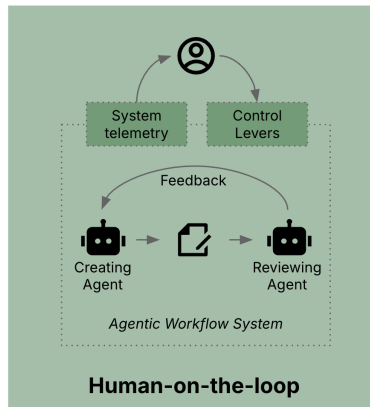
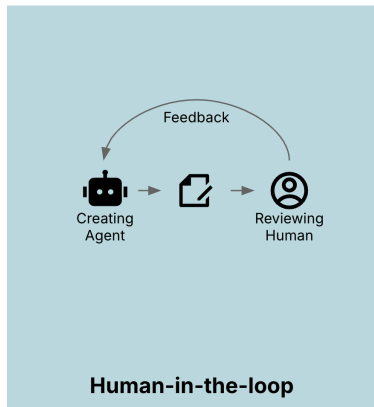
- Teams nutzen Agenten
 - Teams können noch mehr Wertorientiert arbeiten
- Agenten benötigen Wissen um zu arbeiten
 - Wissen teilen und bereitstellen wird relevanter
- Es wird Aufgaben geben die Agententeams lösen können
 - Agententeams müssen aufgebaut und koordiniert werden



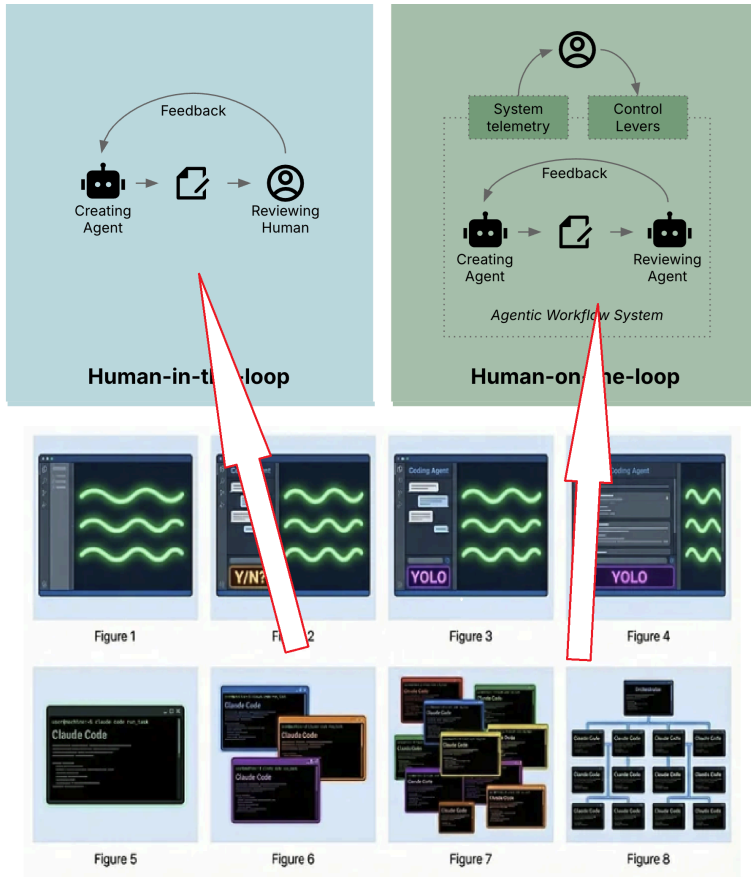
Wird der Mensch abgelöst?

- Software ist für den Menschen gemacht!
- Dokumentation ist für den Menschen gemacht!

Nein, aber was wird seine Aufgabe in der Zukunft sein?



Human in the Loop?



- Ich denke es ist keine Evolution sondern verschiedene Anwendungsfälle benötigen mehr oder weniger Agenten
- Umso mehr der Mensch eingebunden ist um so zielgenauer kann man arbeiten
- Coding Agenten: Claude Code, Github Copilot, Codex
- Agentenschwärme: Gastown, Get Shit Done (GSD), Claude Agents

Ziele der Firma (meiner Ansicht nach)



Commodity - Leistungsfähigkeit erreichen

- Effizienzgewinne der AI nutzen mit gleichbleibender Qualität



Bedienen neuer Aufgaben und Rollen im Enterprise Context

- AI Agenten Virtuos nutzen um damit Prozesse zu gestalten
- AI Agenten Schwärme aufbauen und Orchestrieren können

Mögliche Auswirkungen auf unser Berufsfeld

1. Softwareentwicklung wird **niederschwelliger**, es wird mehr Software geben
2. Es wird relevanter, Agenten zu **beherrschen** und das Wissen benötigte Wissen in den Kontext zu kriegen
3. Die Rolle verschiebt sich: **Code-Schreiber → Architekt, Steuermann, Qualitaetssicherer, UX Designer, Dev/Ops Engineer, AI Agent Engineer**
4. Oder ein Team besteht ausschließlich aus **UX Designer, AI Agent Engineer und PO**
5. AI Agenten können **100% des Codes erzeugen**, es liegt an uns, dass kein **schlechter Code released** wird.

Aber auch: **Human Coding bleibt relevant**

Grundlegendes Credo

- Der Entwickler wird zum **Boss** des Agenten, nicht umgekehrt
- Man ist **verantwortlich** für das was man commiten lässt
- Verstärkung bestehender Fähigkeiten
 - Architektur
 - UX
 - Softwareentwicklungsprozessmethodik
 - Fullstackentwicklung
 - Betrieb von Software



Auswirkungen auf Softwareentwicklungsprozess

Chancen

- Mehr Konsistenz
- Mehr Qualität
- Mehr Fokus auf Werterzeugung
- Mehr Fokus auf Qualitätssicherung

Neu zu erlernende Fähigkeiten

- Agenten zielgerichtet einsetzen und steuern können
- 20% → 100% Coding durch Agenten (**Ziel dieser Reihe**)
- Agentenschwärme bauen und monitoren

Ziele und Inhalte der nächsten Einheiten

Ziele

- Generische Vorgehensweisen zum Umgang mit Agenten lernen
- Coding mit bis zu 3 Agenten gleichzeitig
- Den eigenen Entwicklungsprozess verbessern

Inhalte

- Verständnis von LLMs zum Agenten
- Verständnis über Kontextmanagement erlangen
- Agents.md, (Sub)Agenten und Skills für die eigene Zielstellung nutzbar machen
- Am Beispiel von Github Copilot CLI (IDE unabhängig)

Was kann man schon heute tun

- Ausprobieren von skills, am besten die von obra

```
npx skills add https://github.com/obra/superpowers --skill using-superpowers
```

- brainstorming, plan, execution

Gerne am Beispielprojekt



Brainstorming & Planning mit dem Agenten

Anforderung → Brainstorming → Spec → Plan → Execution

1. **Brainstorming:** Anforderung verstehen, Optionen erkunden
2. **Spec:** Technische Spezifikation schreiben
3. **Plan:** Aufgaben in 5-20min Tasks zerlegen (TDD)
4. **Execution:** Task fuer Task umsetzen mit Entwickler-Checkpoints

Demo: Vibecoding vs Plan, Execute - Gleiche Aufgabe, zwei Ansaetze

Aufgabe: Zusammen mit der Gruppe bestimmmen

	Vibe Coding	Agentic Engineering
Input	"Bau mir X"	Spec → Plan → TDD
Designentscheidungen	Agent entscheidet	Entwickler entscheidet

Modelle und ihre Staerken

Aufgabe	Empfohlenes Modell	Warum
Planning	Claude Opus/Sonnet 4.6	Beste Instruktionsbefolgung
Coding	Gemini 1.5 Pro, Claude Sonnet 4.6	Schnell + präzise

Kosten im Griff behalten

- **Premium-Requests:** Entstehen bei Nutzung leistungsstaerkerer Modelle
- **Einsparen durch:**
 - Richtiges Modell fuer die richtige Aufgabe
 - Guten Kontext (weniger Iterationen = weniger Requests)
 - Standard-Modelle fuer einfache Aufgaben
- **Investition:** Bessere Methodik → weniger Nacharbeit → weniger Kosten

Quellen & Weiterlesen — Modelle und Kosten

- [GitHub Copilot Plans & Pricing](#) — Offizielle Uebersicht aller Plaene mit Premium-Request-Kontingenten und Preisen
- [GitHub Docs: Supported AI Models](#) — Welche Modelle in Copilot verfuegbar sind und ihre Premium-Request-Multiplier
- [GitHub Docs: Model Comparison](#) — Detaillierter Vergleich der Copilot-Modelle nach Staerken und Anwendungsfall
- [Microsoft Tech Community: Choosing the Right Model in GitHub Copilot](#) — Praxisleitfaden zur Modellwahl fuer Entwickler
- [NxCode: Claude Sonnet 4.6 vs Opus 4.6](#) — Detailvergleich: Benchmarks, Kosten, Empfehlung fuer welche Aufgaben

Quellen & Weiterlesen — Blick in die Zukunft

- [thoughtworks Preparing your team for the agentic software development life cycle](#)
- [Jensen Huang: "The programming language is human" \(Tom's Hardware, 2024\)](#) — NVIDIA CEO ueber AI und Programmierung
- [Bill Gates: "Best time to learn to code" \(Windows Central, 2025\)](#) — Gegenperspektive: AI als Werkzeug, nicht Ersatz
- [Andrej Karpathy: Agentic Engineering \(X/Twitter, 2026\)](#) — Karpathys Vision: Orchestrierung statt direktes Coden
- [Gartner: 90% of enterprise developers will use AI by 2028 \(April 2024\)](#) — Marktprognose mit Warnung vor unkontrollierter Nutzung

Quellen & Weiterlesen — Agentic Engineering

- [Lilian Weng: "LLM Powered Autonomous Agents" \(2023\)](#) — Architektur-Übersicht: Planning, Memory, Tool Use als drei Subsysteme
- [Andrej Karpathy: "Vibe Coding" \(X/Twitter, Feb 2025\)](#) — Ursprung des Begriffs "Vibe Coding"
- [Andrej Karpathy: Software is Changing \(X/Twitter, Feb 2026\)](#) — Karpathys Begriff "Agentic Engineering"
- [Thoughtworks: "Preparing your team for the agentic software development life cycle"](#) — Spec-Driven Ansätze im Vergleich
- [DEV Community: "The AI Coding Workflow That Actually Works: Separate Planning from Execution"](#) — Praxis-Perspektive auf Agenten-Ansätze

Quellen & Weiterlesen — Veraenderte Arbeitsweise

- [Andrej Karpathy: "You are orchestrating agents" \(X/Twitter, 2026\)](#) — Die neue Rolle: Steuermann statt Tipper
- [Thoughtworks: Preparing your team for the agentic SDLC](#) — Teamstruktur und Rollen im agentic Umfeld
- [GitHub Blog: Quantifying GitHub Copilot's impact on developer productivity](#) — Entwickler 55,8% schneller mit AI-Unterstützung
- [InfoQ: From Prompts to Production — a Playbook for Agentic Development](#) — Praxis-Playbook fuer Teams

Quellen & Weiterlesen — Demo und Ansaetze im Vergleich

- [Andrej Karpathy: Vibe Coding \(X/Twitter, Feb 2025\)](#) — Ursprung des Begriffs: Coding im Flow ohne Code-Verstaendnis
- [The New Stack: Vibe Coding is Passé](#) — Warum Vibe Coding an Grenzen stoest und Agentic Engineering folgt
- [Dev.to: Separate Planning from Execution — The AI Coding Workflow That Actually Works](#) — Praxis-Vergleich beider Ansaetze mit konkretem Workflow
- [GitHub Blog: Test-Driven Development with GitHub Copilot](#) — TDD als Grundlage fuer nachvollziehbares Agentic Engineering
- [Glide Blog: What is Agentic Engineering](#) — Karpathys Definition und Abgrenzung zu Vibe Coding